

Proyecto 2: Proyecto Colaborativo e Informe

Implementación colaborativa de nuevas funcionalidades mediante flujo de trabajo distribuido con Git y GitHub, sobre los repositorios UOH-Survivors y bird-flappy

Profesor: Felipe Gómez

Integrante: Arnaldo González

RUT: 21.457.660-0

Integrante: Felipe Scanda

RUT: 21.585.645-3

Fecha: 24 de julio de 2026

Repositorio UOH-Survivors: <https://github.com/FelipeScanda/UOH-Survivors>

Repositorio bird-flappy: <https://github.com/nalytox/bird-flappy>

PR en UOH-Survivors: <https://github.com/FelipeScanda/UOH-Survivors/pull/2>

PR en bird-flappy: <https://github.com/nalytox/bird-flappy/pull/2>

Índice de Contenidos

1. Introducción	1
2. Organización del trabajo	1
2.1. Integrantes y roles	1
2.2. Contribución asignada a cada integrante	1
2.3. Coordinación	1
3. Descripción de los proyectos originales	1
3.1. UOH-Survivors	1
3.2. bird-flappy (Flappy Inverso)	2
4. Issues creados	2
4.1. Issue en UOH-Survivors: RAM Boomerang	2
4.2. Issue en bird-flappy: selección de dificultad	4
5. Proceso de desarrollo	5
5.1. Contribución de Arnaldo en UOH-Survivors	5
5.2. Contribución de Felipe en bird-flappy	6
6. Pull requests	7
6.1. PR de Arnaldo en UOH-Survivors	7
6.2. PR de Felipe en bird-flappy	8
7. Proceso de revisión	9
7.1. Revisión del PR de Arnaldo (mantenedor: Felipe)	9
7.2. Revisión del PR de Felipe (mantenedor: Arnaldo)	10
8. Integración de las contribuciones	10
9. Análisis del historial	12
10. Reflexión individual	14
10.1. Arnaldo González	14
10.2. Felipe Scanda	14
11. Conclusiones	15
12. Declaración de uso de herramientas de inteligencia artificial	15

Índice de Figuras

1. Issue “Agrega arma RAM Boomerang con evolución Segmentation Fault” en UOH-Survivors.	3
2. Respuesta de aprobación del mantenedor (Felipe Scanda) sobre el issue del RAM Boomerang.	3
3. Issue “Selección de dificultad” en bird-flappy.	4

4.	Respuesta de aprobación del mantenedor (Arnaldo González) sobre el issue de selección de dificultad.	5
5.	Rama feature/arma-ram-boomerang con sus commits.	6
6.	Rama feature/dificultad-ui con sus commits.	7
7.	Pull request de Arnaldo hacia UOH-Survivors.	7
8.	Sección Files changed del pull request del RAM Boomerang.	8
9.	Pull request de Felipe hacia bird-flappy.	8
10.	Sección Files changed del pull request de selección de dificultad.	9
11.	Comentarios de revisión sobre el pull request del RAM Boomerang.	9
12.	Comentarios de revisión y solicitud de cambios sobre el pull request de selección de dificultad.	10
13.	Aprobación e integración del pull request del RAM Boomerang.	11
14.	Issue del RAM Boomerang cerrado tras la integración.	11
15.	Aprobación e integración del pull request de selección de dificultad.	11
16.	Issue de selección de dificultad cerrado tras la integración.	11
17.	Salida de <code>git remote -v, git branch -all y git log -graph -oneline -all</code> en UOH-Survivors.	12
18.	Salida de <code>git log</code> con formato de autoría en UOH-Survivors.	13
19.	Salida de <code>git remote -v, git branch -all y git log -graph -oneline -all</code> en bird-flappy.	13
20.	Salida de <code>git log</code> con formato de autoría en bird-flappy.	14

1. Introducción

Este informe documenta el desarrollo del Proyecto 2 del curso Introducción a los Repositorios de Código Distribuido, cuyo objetivo fue aplicar un flujo de trabajo distribuido y colaborativo utilizando Git y GitHub sobre los proyectos individuales desarrollados en el Proyecto 1. El trabajo se realizó en pareja entre Arnaldo González y Felipe Scanda, cada uno manteniendo su propio repositorio y actuando como colaborador externo en el repositorio de su compañero.

Los dos proyectos involucrados son **UOH-Survivors**, un videojuego tipo roguelike desarrollado por Felipe Scanda, y **bird-flappy (Flappy Inverso)**, un videojuego desarrollado por Arnaldo González. Sobre cada uno de estos repositorios se llevó a cabo un ciclo completo de colaboración: creación de un issue, fork del repositorio, desarrollo en una rama separada, apertura de un pull request, revisión formal por parte del mantenedor, corrección de los cambios solicitados, y finalmente la aprobación e integración de la contribución.

2. Organización del trabajo

2.1. Integrantes y roles

- **Arnaldo González:** mantenedor del repositorio bird-flappy, y colaborador en el repositorio UOH-Survivors de Felipe Scanda.
- **Felipe Scanda:** mantenedor del repositorio UOH-Survivors, y colaborador en el repositorio bird-flappy de Arnaldo González.

2.2. Contribución asignada a cada integrante

Arnaldo propuso e implementó una nueva arma para UOH-Survivors: el *RAM Boomerang*, un arma temática de computación con una evolución llamada *Segmentation Fault*. Felipe propuso e implementó una pantalla de selección de dificultad para bird-flappy, permitiendo al jugador elegir entre tres niveles de dificultad antes de comenzar la partida.

2.3. Coordinación

La coordinación entre ambos integrantes se realizó íntegramente a través de las herramientas de GitHub: issues para proponer y justificar cada mejora, comentarios de revisión sobre líneas específicas del código, y el hilo de discusión de cada pull request para acordar correcciones. Ambos flujos de colaboración se desarrollaron en paralelo durante el período de trabajo asignado.

3. Descripción de los proyectos originales

3.1. UOH-Survivors

UOH-Survivors es un videojuego desarrollado por Felipe Scanda, inspirado en *Vampire Survivors*, un roguelike de disparos por oleadas donde el objetivo es sobrevivir la mayor cantidad de tiempo posible. Está

desarrollado en Python 3.12 con Pygame 2.6, y su estructura organiza el código en los módulos `entities/` (jugador, enemigos, proyectiles, gemas de experiencia, orbe giratorio) e `items/` (ítems recolectables).

Antes de la colaboración, el repositorio contaba con las siguientes funcionalidades: movimiento del jugador, disparo automático de proyectiles hacia el enemigo más cercano, un arma de orbe giratorio que se obtiene recolectando ítems, sistema de subida de nivel con tres mejoras seleccionables (daño, velocidad de ataque, velocidad de movimiento), tres tipos de enemigos con dificultad creciente en el tiempo, selección de skin del personaje, y los tres estados de juego (menú, pausa, game over). El README del propio proyecto declaraba el estado del juego como jugable pero con desarrollo pendiente, listando explícitamente la incorporación de nuevas armas como una de las mejoras futuras deseadas.

3.2. bird-flappy (Flappy Inverso)

bird-flappy es un videojuego desarrollado por Arnaldo González, una variación de *Flappy Bird* en la que se invierte el rol habitual: el jugador no controla al ave (Barranquín, un loro trichahue con temática de la UOH), sino que controla el barranco (la compuerta) que debe encajar con el vuelo autónomo e impredecible del ave. También desarrollado en Python con Pygame, organiza su lógica en `bird.py` (física de vuelo del ave, basada en un oscilador amortiguado), `gate.py` (la compuerta controlada por el jugador) y `main.py` (bucle principal y estados del juego).

Antes de la colaboración, el repositorio contaba con: física de vuelo autónoma del ave, control de la compuerta con teclado, detección de colisiones, persistencia del mejor puntaje en un archivo local, un fondo procedural de precordillera, y los tres estados de juego (inicio, jugando, game over), con una única configuración de dificultad fija que progresaba con el puntaje.

4. Issues creados

4.1. Issue en UOH-Survivors: RAM Boomerang

Arnaldo creó el issue “*Agrega arma RAM Boomerang con evolución Segmentation Fault*” en el repositorio de Felipe. La propuesta consistió en incorporar una nueva arma temática de computación: un stick de RAM que se lanza como boomerang, dañando enemigos al ir y al volver, con una evolución que provoca un daño masivo a todos los enemigos en pantalla mediante un evento visual que simula el crasheo de la aplicación. Los criterios de aceptación definidos fueron: que el arma apareciera como opción seleccionable, que infligiera daño en ambos tramos del recorrido con retroalimentación visual de daño, que la evolución se activara correctamente, que el evento de evolución afectara solo a enemigos visibles en pantalla sin congelar el juego real, y que el juego siguiera ejecutándose sin errores.

Felipe, como mantenedor, revisó y aprobó la propuesta antes de que comenzara la implementación.

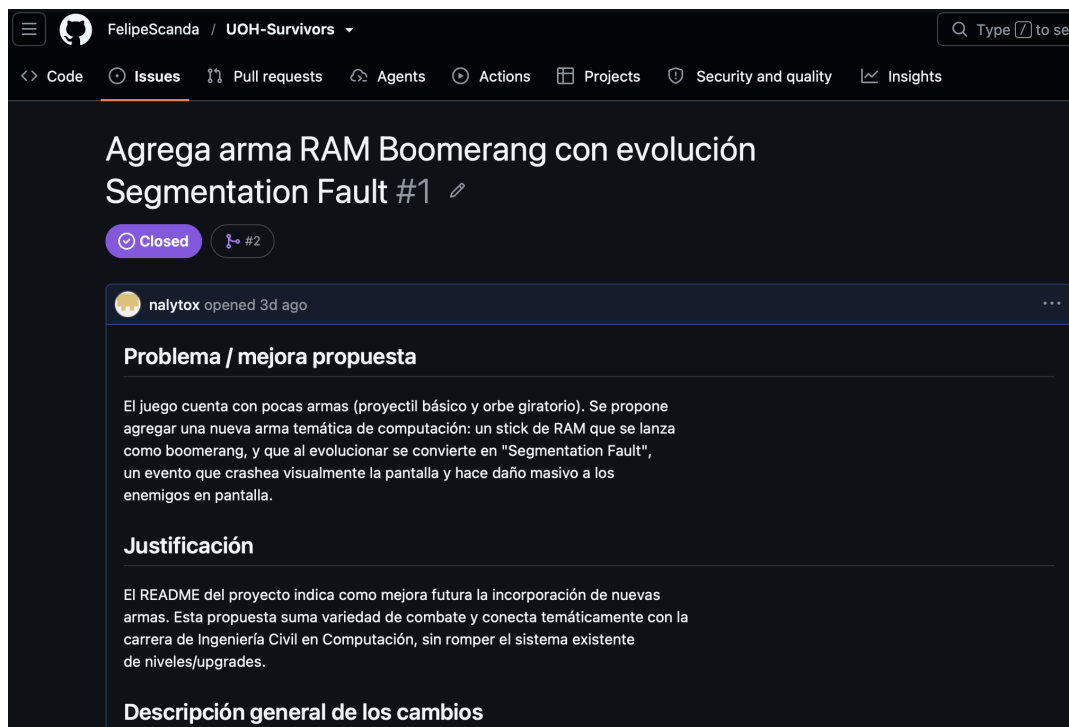


Figura 1: Issue “Agrega arma RAM Boomerang con evolución Segmentation Fault” en UOH-Survivors.



Figura 2: Respuesta de aprobación del mantenedor (Felipe Scanda) sobre el issue del RAM Boomerang.

4.2. Issue en bird-flappy: selección de dificultad

Felipe creó el issue “*Selección de dificultad*” en el repositorio de Arnaldo. El problema planteado fue que el juego contaba con una única configuración de dificultad, y aunque la velocidad y el tamaño del hueco cambiaban progresivamente con el puntaje, el jugador no podía elegir una dificultad inicial acorde a su experiencia. La propuesta fue agregar una pantalla de selección de dificultad con tres opciones (fácil, normal y difícil), cada una modificando la velocidad inicial del barranco, el tamaño inicial del hueco y la intensidad de la progresión. Los criterios de aceptación exigían que la dificultad fuera seleccionable antes de iniciar, que afectara realmente el comportamiento del juego, que se mostrara en pantalla durante la partida, que se conservara o pudiera cambiarse al reiniciar, y que el README documentara la nueva funcionalidad.

Arnaldo, como mantenedor, aprobó la propuesta antes de que Felipe comenzara la implementación.



Figura 3: Issue “Selección de dificultad” en bird-flappy.

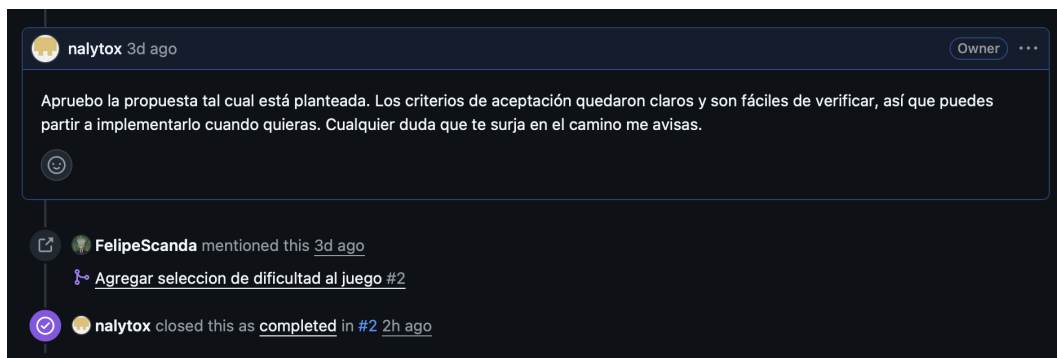


Figura 4: Respuesta de aprobación del mantenedor (Arnaldo González) sobre el issue de selección de dificultad.

5. Proceso de desarrollo

5.1. Contribución de Arnaldo en UOH-Survivors

Arnaldo realizó un fork de UOH-Survivors, lo clonó localmente y configuró el remoto upstream apuntando al repositorio original de Felipe:

```
git clone https://github.com/nalytox/UOH-Survivors.git
cd UOH-Survivors
git remote add upstream https://github.com/FelipeScanda/UOH-Survivors.git
```

El desarrollo se realizó íntegramente en la rama `feature/arma-ram-boomerang`, creada a partir de `main`. La contribución se dividió en commits con separación lógica clara:

1. “*Agrega clase RAMBoomerang con comportamiento de ida y vuelta*”: se agregó `entities/ram_boomerang.py`, con la lógica de movimiento del boomerang (tramo de ida, distancia máxima, tramo de vuelta) y su representación visual, incluyendo el cambio de apariencia al regresar “dañado”.
2. “*Integra RAM Boomerang al ciclo de juego (spawn, colisión, dibujo)*”: se modificó `main.py` para lanzar el arma automáticamente, detectar colisiones con enemigos y dibujarla en pantalla.
3. “*Agrega opción RAM Boomerang al menú de mejoras de nivel*”: se incorporó el arma como opción seleccionable al subir de nivel.
4. “*Agrega evolución Segmentation Fault con daño masivo y efectos visuales*”: se agregó `entities/-segfault_event.py` y su integración, activando un evento periódico de daño de área con retroalimentación visual (overlay, mensajes de error simulados) al alcanzar el nivel 3 del arma.
5. “*Corrige daño fuera de pantalla, normaliza dirección del boomerang y ajusta colisión a hitbox circular*”: commit de corrección de errores encontrados durante pruebas propias antes de solicitar la revisión, detallado en la Sección 7.

Antes de abrir el pull request, se sincronizó la rama con los cambios más recientes del repositorio original:


```
git fetch upstream
git switch main
git merge upstream/main
git switch feature/arma-ram-boomerang
git merge main
```

No se registraron cambios nuevos en upstream/main al momento de sincronizar, por lo que no fue necesario resolver conflictos.

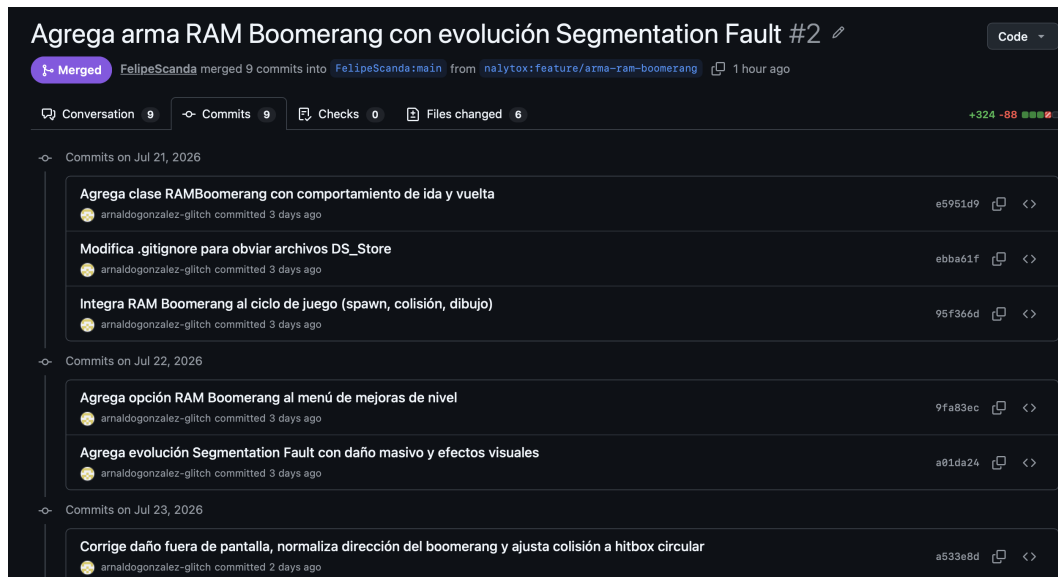


Figura 5: Rama feature/arma-ram-boomerang con sus commits.

5.2. Contribución de Felipe en bird-flappy

Felipe realizó un fork de bird-flappy, lo clonó localmente y configuró el remoto upstream apuntando al repositorio original de Arnaldo:

```
git clone https://github.com/FelipeScanda/bird-flappy.git
cd bird-flappy
git remote add upstream https://github.com/nalytox/bird-flappy.git
git switch -c feature/dificultad-ui
```

Sobre la rama feature/dificultad-ui, Felipe implementó el nuevo estado de juego STATE_DIFFICULTY, la función draw_difficulty_selection para renderizar la pantalla de selección, la navegación entre las tres dificultades con las flechas o W/S, y la propagación de la dificultad seleccionada hacia la clase Gate, que pasó a recibir la configuración correspondiente (velocidad inicial, tamaño del hueco y su progresión) desde un diccionario DIFFICULTIES agregado en settings.py.

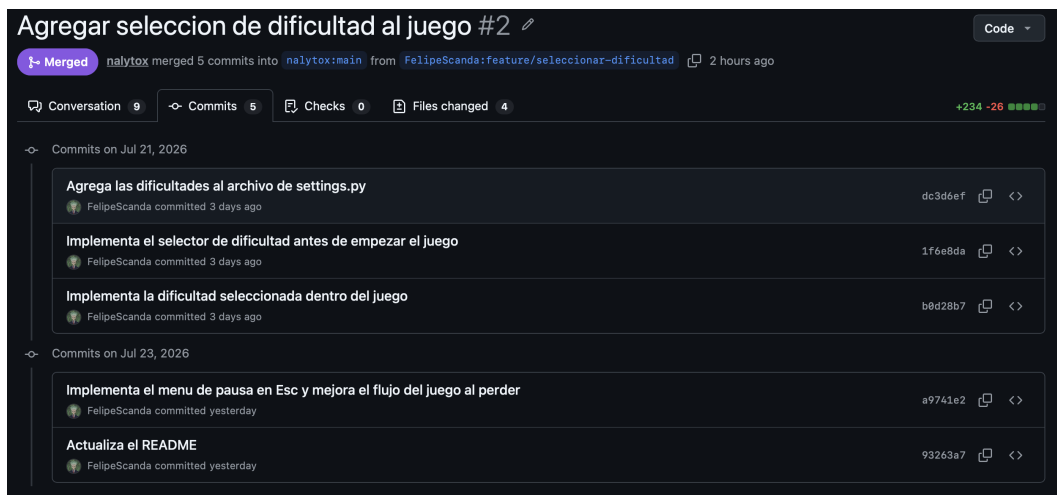


Figura 6: Rama feature/dificultad-ui con sus commits.

6. Pull requests

6.1. PR de Arnaldo en UOH-Survivors

Arnaldo subió su rama al fork y abrió el pull request hacia FelipeScanda/UOH-Survivors:main, vinculado al issue mediante Closes #1. El PR describía los tres archivos modificados (`entities/ram_boomerang.py`, `entities/segfault_event.py` y `main.py`) y las instrucciones para probar el cambio: ejecutar `python main.py`, subir de nivel eligiendo la opción del RAM Boomerang tres veces, y verificar tanto el comportamiento de ida y vuelta del arma como la activación del evento de evolución.

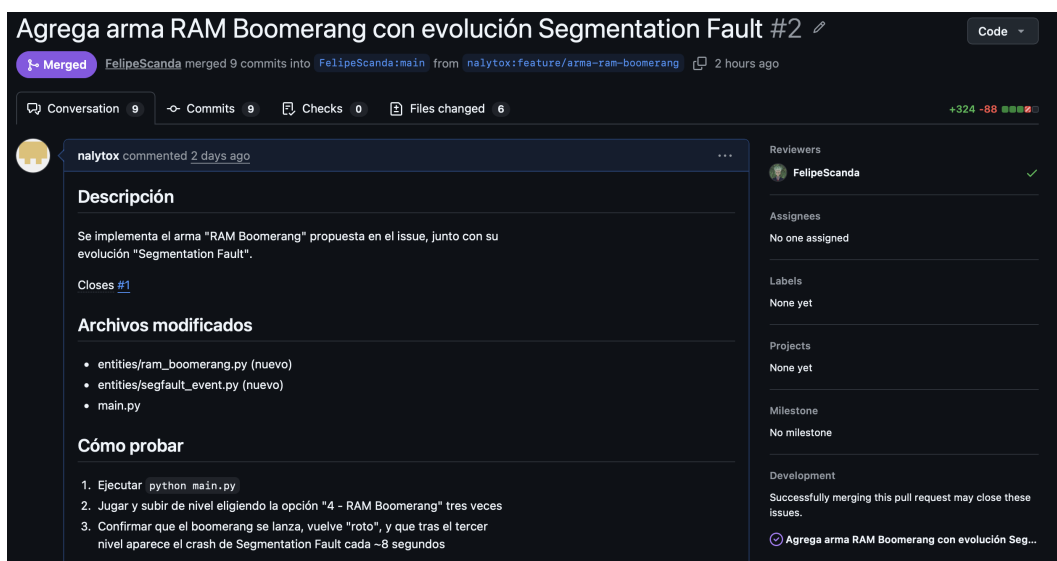


Figura 7: Pull request de Arnaldo hacia UOH-Survivors.

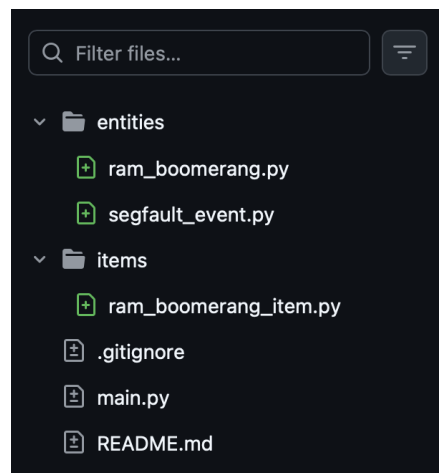


Figura 8: Sección Files changed del pull request del RAM Boomerang.

6.2. PR de Felipe en bird-flappy

Felipe subió su rama al fork y abrió el pull request hacia nalytox/bird-flappy:main, vinculado igualmente al issue de selección de dificultad. El PR describía los archivos modificados (`main.py`, `settings.py` y `gate.py`) y las instrucciones para probar el cambio, indicando que debía navegarse la nueva pantalla de dificultad y verificar que cada modo afectara efectivamente la velocidad y el tamaño del hueco durante la partida.

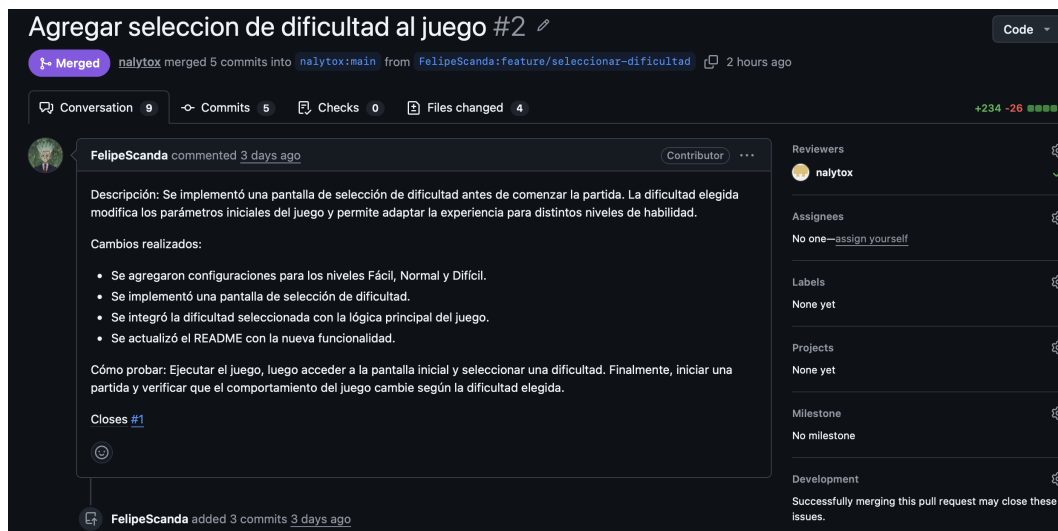


Figura 9: Pull request de Felipe hacia bird-flappy.

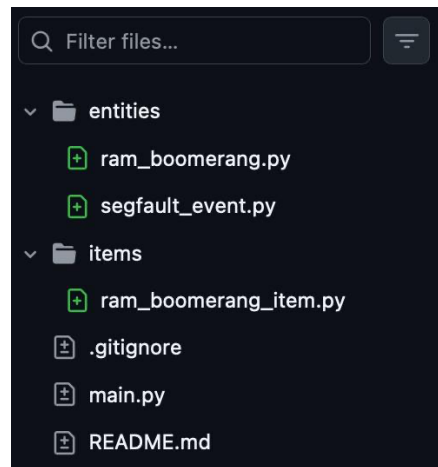


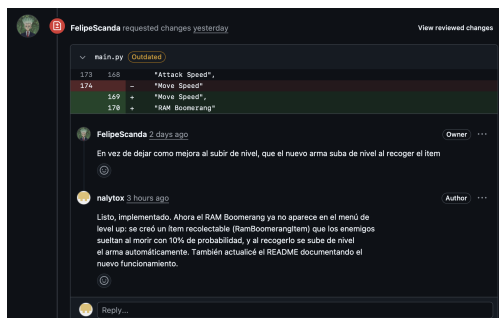
Figura 10: Sección Files changed del pull request de selección de dificultad.

7. Proceso de revisión

7.1. Revisión del PR de Arnaldo (mantenedor: Felipe)

Felipe revisó el pull request del RAM Boomerang probando el juego localmente. Antes de abrir el PR, Arnaldo ya había detectado y corregido de forma proactiva tres problemas durante sus propias pruebas: el evento de evolución dañaba a todos los enemigos del mapa en lugar de solo a los visibles en pantalla (lo cual contradecía el propio criterio de aceptación del issue), la dirección del boomerang no se normalizaba, pudiendo dejarlo indefinidamente en pantalla en un caso límite, y la colisión utilizaba un rectángulo sin rotación que no calzaba visualmente con el sprite rotado. Estas tres correcciones quedaron documentadas en un commit de bugfix independiente antes de la revisión formal.

Además, durante una segunda ronda de trabajo, Felipe solicitó un cambio en el diseño del arma: que el RAM Boomerang subiera de nivel al recolectar un ítem en el mapa, en lugar de ocupar un espacio en el menú de mejoras al subir de nivel del personaje. Arnaldo implementó este cambio creando la clase RamBoomerangItem en items/, un ítem rectangular verde que los enemigos sueltan con 10% de probabilidad al morir, retirando la opción del menú de subida de nivel y trasladando la lógica de subida de nivel del arma al momento de recolectar el ítem. Este cambio se documentó también en el README del proyecto.



(a) Primer comentario de revisión.

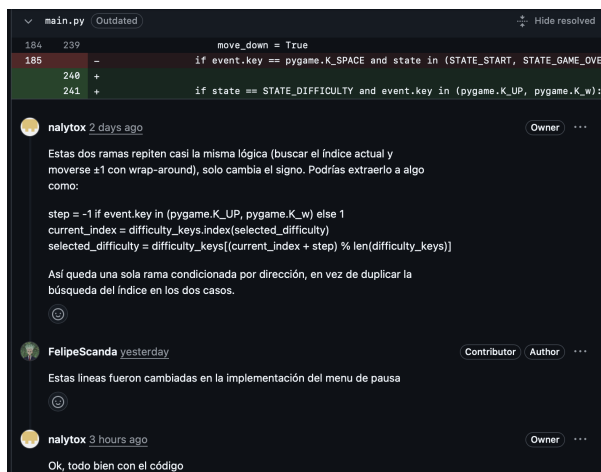


(b) Segundo comentario de revisión.

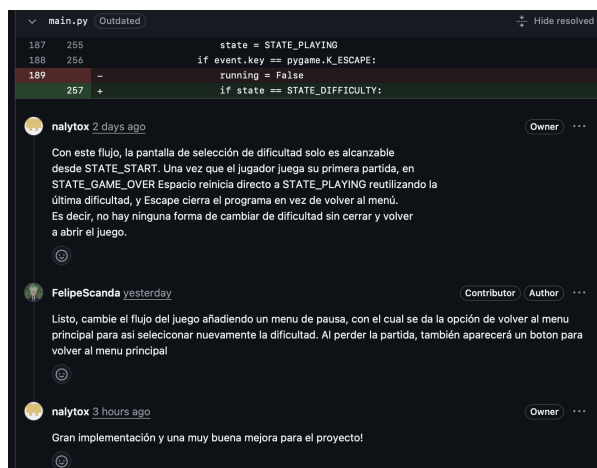
Figura 11: Comentarios de revisión sobre el pull request del RAM Boomerang.

7.2. Revisión del PR de Felipe (mantenedor: Arnaldo)

Arnaldo revisó el pull request de selección de dificultad probando el juego localmente y comparando el código contra la versión original del repositorio. Se dejaron comentarios sobre líneas específicas del código, señalando que las variables de movimiento de la compuerta se activaban también durante la navegación de la nueva pantalla de dificultad (lo que podía arrastrar un movimiento no intencionado hacia la partida), y que la nueva pantalla de selección utilizaba valores de posición en píxeles fijos en lugar de proporciones relativas al alto de la ventana, a diferencia del resto de las pantallas del juego. Como solicitud formal de cambios (*Request changes*), se indicó que el README no documentaba aún la nueva pantalla de dificultad ni sus controles, lo cual era parte explícita de los criterios de aceptación del issue original.



(a) Primer comentario de revisión.



(b) Segundo comentario de revisión.

Figura 12: Comentarios de revisión y solicitud de cambios sobre el pull request de selección de dificultad.

Felipe corrigió el PR de bird-flappy incorporando la documentación faltante en el README y respondiendo los comentarios de revisión de Arnaldo directamente en el hilo del pull request. Arnaldo, por su parte, aplicó las correcciones al PR de UOH-Survivors en la misma rama *feature/arma-ram-boomerang*, subiendo los commits de corrección con `git push` y respondiendo los comentarios de revisión de Felipe indicando cómo fueron resueltos.

8. Integración de las contribuciones

Ambos pull requests fueron aprobados por su respectivo mantenedor una vez verificadas las correcciones solicitadas, e integrados a la rama *main* de cada repositorio original. Los issues asociados quedaron cerrados automáticamente mediante la referencia `Closes #1` incluida en la descripción de cada pull request.

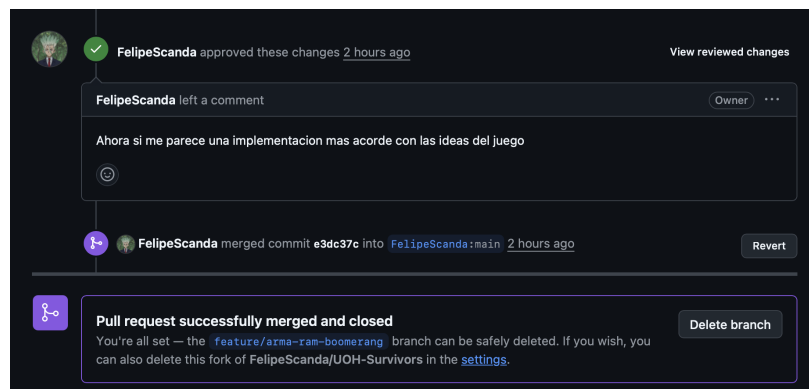


Figura 13: Aprobación e integración del pull request del RAM Boomerang.

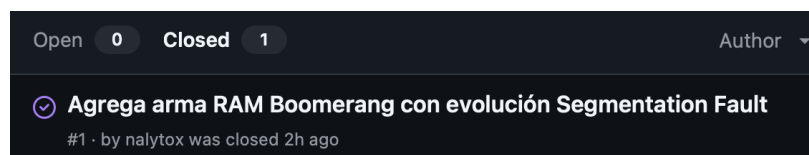


Figura 14: Issue del RAM Boomerang cerrado tras la integración.

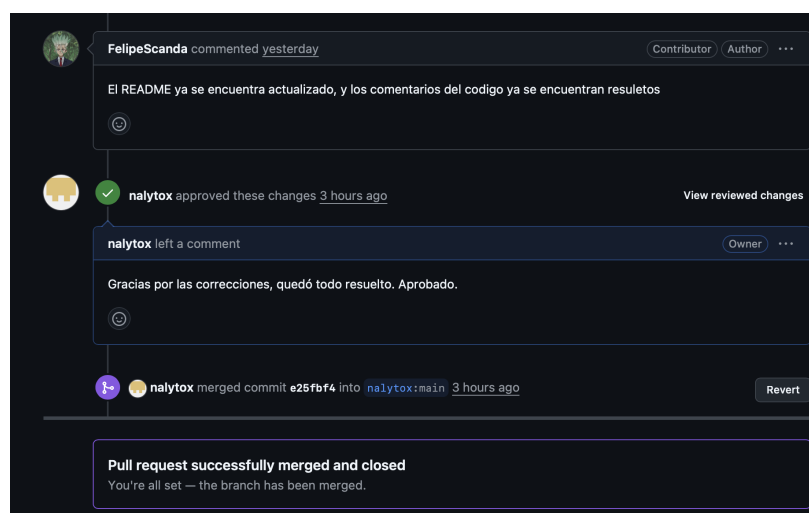


Figura 15: Aprobación e integración del pull request de selección de dificultad.

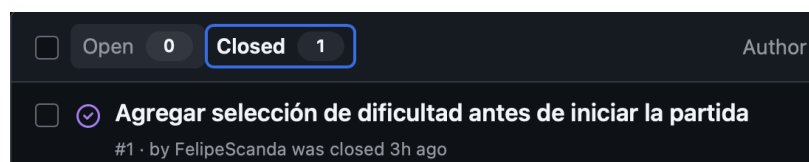


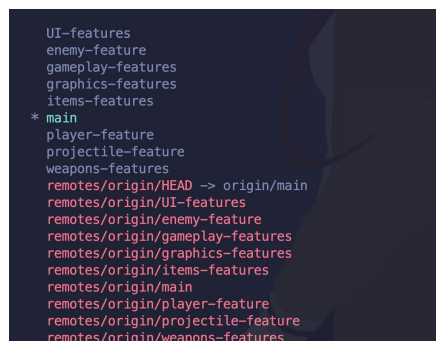
Figura 16: Issue de selección de dificultad cerrado tras la integración.

9. Análisis del historial

Tras la integración de ambos pull requests, se extrajo evidencia del estado final de cada repositorio mediante los siguientes comandos, ejecutados sobre la rama main ya actualizada de cada uno:

```
git remote -v
git branch --all
git log --graph --oneline --all
git log --pretty=format:@"%h | %ad | %an | %ae | %s" --date=short
```

En UOH-Survivors, el historial muestra los cinco commits de la contribución del RAM Boomerang integrados sobre el historial previo del Proyecto 1, sin alterar los commits originales de Felipe. La autoría de cada commit, verificable mediante el correo y nombre asociados, corresponde efectivamente a la cuenta de GitHub de Arnaldo.



(a) Salida de `git branch -all`.



(b) Salida de `git log -graph -oneline -all`.



(c) Salida de `git remote -v`.

Figura 17: Salida de `git remote -v`, `git branch -all` y `git log -graph -oneline -all` en UOH-Survivors.

e3dc37c	2026-07-24	Felipe Scanda	felipescanda@gmail.com	Merge pull request #2 from nalytox/feature/arma-ram-boomerang
816be55	2026-07-24	arnalxytox	arnaldo.gonzalez@pregrado.uoh.cl	Documenta el sistema de items del RAM Boomerang en el README
f860479	2026-07-24	arnalxytox	arnaldo.gonzalez@pregrado.uoh.cl	Quita RAM Boomerang del menu de level up
8a9b7ed	2026-07-24	arnalxytox	arnaldo.gonzalez@pregrado.uoh.cl	Agrega item recolectable RamBoomerangItem
a533e8d	2026-07-23	arnalxytox	arnaldo.gonzalez@pregrado.uoh.cl	Corrige daño fuera de pantalla, normaliza dirección del boomerang y ajusta colisión a hitbox circular
a01da24	2026-07-22	arnalxytox	arnaldo.gonzalez@pregrado.uoh.cl	Agrega evolución Segmentation Fault con daño masivo y efectos visuales
9fa83ec	2026-07-22	arnalxytox	arnaldo.gonzalez@pregrado.uoh.cl	Agrega opción RAM Boomerang al menú de mejoras de nivel
95f366d	2026-07-21	arnalxytox	arnaldo.gonzalez@pregrado.uoh.cl	Integra RAM Boomerang al ciclo de juego (spawn, colisión, dibujo)
ebba61f	2026-07-21	arnalxytox	arnaldo.gonzalez@pregrado.uoh.cl	Modifica .gitignore para obviar archivos DS_Store
e5951d9	2026-07-21	arnalxytox	arnaldo.gonzalez@pregrado.uoh.cl	Agrega clase RAMBoomerang con comportamiento de ida y vuelta
8eb96dc	2026-07-05	Felipe Scanda	felipescanda@gmail.com	Agrega informe en PDF
81f666b	2026-07-05	Felipe Scanda	felipescanda@gmail.com	Agrega instrucciones de juego al README.md
5aa6008	2026-07-05	Felipe Scanda	felipescanda@gmail.com	Actualiza README.md
d5b415f	2026-07-05	Felipe Scanda	felipescanda@gmail.com	Resuelve conflicto entre UI-features y graphics-features
43d6c25	2026-07-05	Felipe Scanda	felipescanda@gmail.com	Mergea rama 'UI-features'
ae053c5	2026-07-05	Felipe Scanda	felipescanda@gmail.com	Modifica el color de la barra de vida y formatea su codigo de main.py
11f8019	2026-07-05	Felipe Scanda	felipescanda@gmail.com	Mueve la posicion de la UI y formatea su codigo de main.py
6a89fbc	2026-07-05	Felipe Scanda	felipescanda@gmail.com	Ajusta el tamaño del sprite del jugador
9a3879d	2026-07-05	Felipe Scanda	felipescanda@gmail.com	Implementa seleccion de skins en menu principal
6478cbe	2026-07-05	Felipe Scanda	felipescanda@gmail.com	Agrega variable para seleccionar skin
6eb84bb	2026-07-05	Felipe Scanda	felipescanda@gmail.com	Mergea rama 'main' en graphics-features
1906ef	2026-07-05	Felipe Scanda	felipescanda@gmail.com	Agrega los sprites del jugador
391dae2	2026-07-04	Felipe Scanda	felipescanda@gmail.com	Elimina el renderizado del gameplay en el menu principal
2ed223f	2026-07-04	Felipe Scanda	felipescanda@gmail.com	Añade interfaz de menu principal, menu de pausa y pantalla de gameover

Figura 18: Salida de `git log` con formato de autoría en UOH-Survivors.

En bird-flappy, de manera análoga, el historial muestra los commits de la contribución de Felipe integrados sobre el historial previo del proyecto, conservando el historial original.

```
(base) arnalytox@MacBook-Pro-de-arnaldo-164 githu % git log --graph --oneline --all
* b0d28b7 (revision-dificultad) Implementa la dificultad seleccionada dentro del juego
* 1fe8da Implementa el selector de dificultad antes de empezar el juego
* dc3ddef Agrega las dificultades al archivo de settings.py
* cb7175d (HEAD -> main, origin/main, origin/HEAD) añadido el pdf real de la entrega
* 017c35b Integra pantallas y pulido final
/
/
/
* f98ce3f (origin/feature/dificultad-ui, feature/dificultad-ui) agregolas pantallas de inicio/fin, mejor puntaje y pulido final
/
/
/
* 807b58f resuelvo el conflicto de dispersion del vuelo: unifica en 60
/
/
/
* 9378996 (origin/feature/barranco-control, feature/barranco-control) ajusto la dispersion del vuelo de Barranquin tras pruebas
* 9f6beeb Ajusta velocidad de control del barranco tras pruebas
* d2b8c71 añado el control con el teclado, físicas de colisiones y sistema puntaje
* d2738ab agrego el barranco, donde este se mueve pero aun sin control del jugador
* 69618dd ajusto dispersion del vuelo de Barranquin (prueba en main)
/
/
/
* 3bfd15d Integra el vuelo de Barranquin en main
/
/
/
* 35f132a (origin/feature/pajaro-random, feature/pajaro-random) implemento el vuelo del pájaro siguiendo una distribución normal
* afb4ade agrego el README
/
/
/
* 564090e Inicializo el proyecto incluyendo la ventana y el fondo
```

(a) Salida de `git log -graph -oneline -all`.

```
(base) arnalytox@MacBook-Pro-de-arnaldo-164 githu % git branch --all
feature/barranco-control
feature/dificultad-ui
feature/pajaro-random
* main
revision-dificultad
remotes/origin/HEAD -> origin/main
remotes/origin/feature/barranco-control
remotes/origin/feature/dificultad-ui
remotes/origin/feature/pajaro-random
remotes/origin/main
```

(b) Salida de `git branch -all`.

```
> git remote -v
origin git@github.com:FelipeScanda/bird-flappy.git (fetch)
origin git@github.com:FelipeScanda/bird-flappy.git (push)
upstream git@github.com:nalytox/bird-flappy.git (fetch)
upstream git@github.com:nalytox/bird-flappy.git (push)
```

(c) Salida de `git remote -v`.Figura 19: Salida de `git remote -v`, `git branch -all` y `git log -graph -oneline -all` en bird-flappy.


```
(base) arnalytox@MacBook-Pro-de-arnaldo-164 githu % git log --all --pretty=format:"%h | %ad | %an | %ae | %s" --date=s
hort
e25fbf4 | 2026-07-24 | Nailen | 114202518+nalytox@users.noreply.github.com | Merge pull request #2 from FelipeScanda/f
eature/seleccionar-dificultad
93263a7 | 2026-07-23 | Felipe Scanda | felipescanda@gmail.com | Actualiza el README
a9741e2 | 2026-07-23 | Felipe Scanda | felipescanda@gmail.com | Implementa el menu de pausa en Esc y mejora el flujo d
el juego al perder
b0d28b7 | 2026-07-21 | Felipe Scanda | felipescanda@gmail.com | Implementa la dificultad seleccionada dentro del juego
1f6e8da | 2026-07-21 | Felipe Scanda | felipescanda@gmail.com | Implementa el selector de dificultad antes de empezar
el juego
dc3d6ef | 2026-07-21 | Felipe Scanda | felipescanda@gmail.com | Agrega las dificultades al archivo de settings.py
cb7175d | 2026-07-06 | arnalytox | arnaldo.gonzalez@pregrado.uoh.cl | añadido el pdf real de la entrega
017c35b | 2026-07-05 | arnalytox | arnaldo.gonzalez@pregrado.uoh.cl | Integra pantallas y pulido final
f98ce3f | 2026-07-05 | arnalytox | arnaldo.gonzalez@pregrado.uoh.cl | agregolas pantallas de inicio/fin, mejor puntaje
y pulido final
807b58f | 2026-07-05 | arnalytox | arnaldo.gonzalez@pregrado.uoh.cl | resuelvo el conflicto de dispersion del vuelo: u
nifica en 60
69610dd | 2026-07-05 | arnalytox | arnaldo.gonzalez@pregrado.uoh.cl | ajusto dispersion del vuelo de Barranquin (prueb
a en main)
9378996 | 2026-07-05 | arnalytox | arnaldo.gonzalez@pregrado.uoh.cl | ajusto la dispersion del vuelo de Barranquin tra
s pruebas
9f6beeb | 2026-07-05 | arnalytox | arnaldo.gonzalez@pregrado.uoh.cl | Ajusta velocidad de control del barranco tras pr
uebas
d2b8c71 | 2026-07-05 | arnalytox | arnaldo.gonzalez@pregrado.uoh.cl | añadido el control con el teclado, físicas de coli
siones y sistema puntaje
d2738ab | 2026-07-05 | arnalytox | arnaldo.gonzalez@pregrado.uoh.cl | agrego el barranco, donde este se mueve pero aun
sin control del jugador
3bfd15d | 2026-07-05 | arnalytox | arnaldo.gonzalez@pregrado.uoh.cl | Integra el vuelo de Barranquin en main
afb4ade | 2026-07-05 | arnalytox | arnaldo.gonzalez@pregrado.uoh.cl | agrego el README
35f132a | 2026-07-05 | arnalytox | arnaldo.gonzalez@pregrado.uoh.cl | implemento el vuelo del pájaro siguiendo una dis
tribucion normal
564090e | 2026-07-05 | arnalytox | arnaldo.gonzalez@pregrado.uoh.cl | Inicializo el proyecto incluyendo la ventana y e
l fondo
```

Figura 20: Salida de `git log` con formato de autoría en bird-flappy.

10. Reflexión individual

10.1. Arnaldo González

Contribuir a un repositorio ajeno implicó primero entender la estructura y las convenciones de código de un proyecto que no diseñé yo, antes de poder agregar algo nuevo sin que se sintiera fuera de lugar. La principal dificultad no fue tanto escribir el arma en sí, sino integrarla de forma consistente con el resto del sistema: reutilizar el mismo patrón de clases con `update/draw`, y hacer que la nueva arma conviviera bien con el sistema de subida de nivel ya existente.

Recibir comentarios sobre mi propio código me sirvió para notar cosas que, viniendo de mí mismo, probablemente no hubiera cuestionado, como el hecho de que el daño masivo de la evolución afectaba a enemigos fuera de pantalla, que técnicamente contradecía lo que yo mismo había definido en el issue. También me hizo confirmar que cambios de diseño (como pasar de subir de nivel por menú a subir de nivel por ítem recolectable) pueden pedirse durante el proceso de revisión y no solo al principio.

La diferencia entre hacer un merge local y aceptar un pull request quedó bastante clara: un merge local no deja rastro de revisión ni de discusión, mientras que el flujo de PR obliga a que exista una instancia formal donde alguien más entienda, cuestione y valide el cambio antes de integrarlo. El flujo de forks y pull requests tiene la ventaja de mantener separado el trabajo experimental del repositorio estable del compañero, aunque agrega la dificultad de mantener sincronizados origin y upstream para no trabajar sobre una base desactualizada.

10.2. Felipe Scanda

La primera dificultad que tuve al contribuir al repositorio de mi compañero fue principalmente el comprender cómo este había sido construido, comprender los commits que mi compañero había realizado anteriormente para llegar al estado actual del repositorio, para luego de eso poder contribuir mi parte. De todas

formas, al ver y revisar el código de mi compañero aprendí a ver su forma de trabajar y de cómo yo puedo integrarme a ella de manera de que pueda contribuir de forma positiva y constructiva a su proyecto.

Al ver los comentarios de mi compañero sobre el código que propuse inicialmente a su repositorio pude comprender de mejor manera sus ideas sobre su propio juego y cómo él esperaba que fueran implementadas, dandome cuenta además de que el flujo de juego podía mejorarse, por ejemplo, al momento de perder, no había una manera de volver a elegir la dificultad, por lo que tuve margen para mejorar mi implementación.

La diferencia entre realizar un merge local y un pull request es básicamente el proceso que implica cada uno. En un merge local, el código a implementar en la rama seleccionada está escrito por uno mismo, lo cual agiliza todo y se hace de manera directa, en cambio, en un pull request, al ser código proveniente de otra persona, se tiene que revisar con detenimiento para asegurarse del correcto funcionamiento del mismo, y que la implementación no vaya a romper otras partes del proyecto o contenga código no deseado, para luego aceptar o rechazar los cambios del pull request, teniendo interacciones con comentarios con la otra persona, los request changes, etc.

La principal ventaja de trabajar en un fork de un proyecto ya hecho es la localidad del repositorio, ya que uno puede trabajar dentro de su mismo fork sin interrumpir el repositorio original, y mandar el pull request una vez se esté seguro de los cambios a realizar. Una dificultad de esto puede ser el hecho de configurar el repositorio origin y el upstream, lo que implica que en el repositorio original se pueda dar el caso de que se sigue avanzando en el proyecto, se realicen cambios y al pullear desde el upstream se interfiera con el avance del fork, teniendo que resolver conflictos y retrasando el desarrollo.

11. Conclusiones

El desarrollo de este proyecto permitió aplicar de manera práctica un flujo de trabajo distribuido completo: desde la propuesta formal de una mejora mediante un issue, pasando por el desarrollo aislado en una rama sobre un fork, hasta la revisión, corrección e integración final mediante pull request. Trabajar en ambas direcciones (como colaborador en un repositorio ajeno y como mantenedor del propio) permitió experimentar las dos caras del proceso: la necesidad de adaptarse a un código y unas convenciones que no son propias al momento de contribuir, y la responsabilidad de evaluar críticamente una contribución externa al momento de revisarla.

Más allá de la mecánica de comandos de Git, el ejercicio deja en evidencia el valor de la revisión de código como instancia de control de calidad: varios de los ajustes realizados en ambos proyectos (la corrección del alcance del daño masivo en el RAM Boomerang, la documentación faltante en el README de bird-flappy) surgieron directamente de la revisión cruzada, y probablemente no se habrían detectado trabajando de forma aislada. En ese sentido, el flujo de issues, forks, ramas y pull requests no solo organiza el trabajo en equipo, sino que mejora la calidad final del software producido.

12. Declaración de uso de herramientas de inteligencia artificial

Se utilizó Claude (Anthropic) como apoyo durante el desarrollo de este proyecto, específicamente en: el diseño e implementación del arma RAM Boomerang y su evolución Segmentation Fault (incluyendo la detección y corrección de errores antes de la revisión formal), el diseño e implementación del sistema de ítem recolectable para la subida de nivel del arma, y Codex (OpenIA) para la implementación de la opción de dificultad para el bird-flappy. Todo el código entregado fue revisado, probado localmente y puede ser explicado por los integrantes del equipo.